STUDENT PERFORMANCE ANALYTICS SYSTEM

A Data Analysis Project using Python, Pandas, and NumPy

1. Project Title & Student Details

- **Project Title:** Student Performance Analytics System
- **Student Name:** Soumya Ranjan Behera
- **College/University Name:** Ravenshaw University
- **Course Name:** Python with AI
- **Institution:** EWB Courses
- **Submission Date:** October 2026

2. Objective

The primary objective of this project is to develop a Python-based data processing and analytics tool to evaluate academic performance data. The system automates the loading of raw student records, processes scores using NumPy and Pandas, applies grading logic, and produces structured analytical reports. Key objectives include:

- Computing individual totals, average scores, and attendance percentages.
- Classifying academic performance into distinct letter grades and pass/fail statuses.
- Performing aggregate class-level analysis to highlight top performers and subject-wise averages.

3. Technologies Used

- **Programming Language:** Python 3.x
- **Core Libraries:**

    - **Pandas:** Used for DataFrame creation, data manipulation, filtering, and structural calculations.
    - **NumPy:** Utilized for vectorized mathematical operations and conditional statistical analysis.
- **Development Environment:** VS Code/ collab

4. Dataset Description

The dataset is structured as a CSV file (`students.csv`) containing 50 student records across various academic departments.

Dataset Schema:

Column Name	Data Type	Description
`Student_ID`	Integer	Unique identifier for each student
`Name`	String	Full name of the student
`Department`	String	Academic stream (e.g., Computer Science, IT, Electrical)
`Math`	Integer	Marks scored in Mathematics (out of 100)
`Science`	Integer	Marks scored in Science (out of 100)
`English`	Integer	Marks scored in English (out of 100)
`Attendance`	Integer	Percentage of total attendance recorded

5. Implementation Details

Step 1: Data Acquisition & Ingestion

The dataset is read from `students.csv` into a Pandas DataFrame (`df`) using `pd.read_csv()`. Initial inspection functions (`df.head()`, `df.info()`) are executed to verify data structure and column data types.

Step 2: Data Processing & Feature Engineering

1. **Total & Average Marks:** Calculated across the subject columns (`Math`, `Science`, `English`) using Pandas `.sum(axis=1)` and `.mean(axis=1)`.
2. **Grading Engine:** A custom Python function maps individual average marks to letter grades (`A`, `B`, `C`, `F`) based on predefined thresholds.
3. **Pass/Fail Evaluation:** Implemented using `np.where()` to assign pass status based on overall percentage criteria and individual subject minimum requirements.

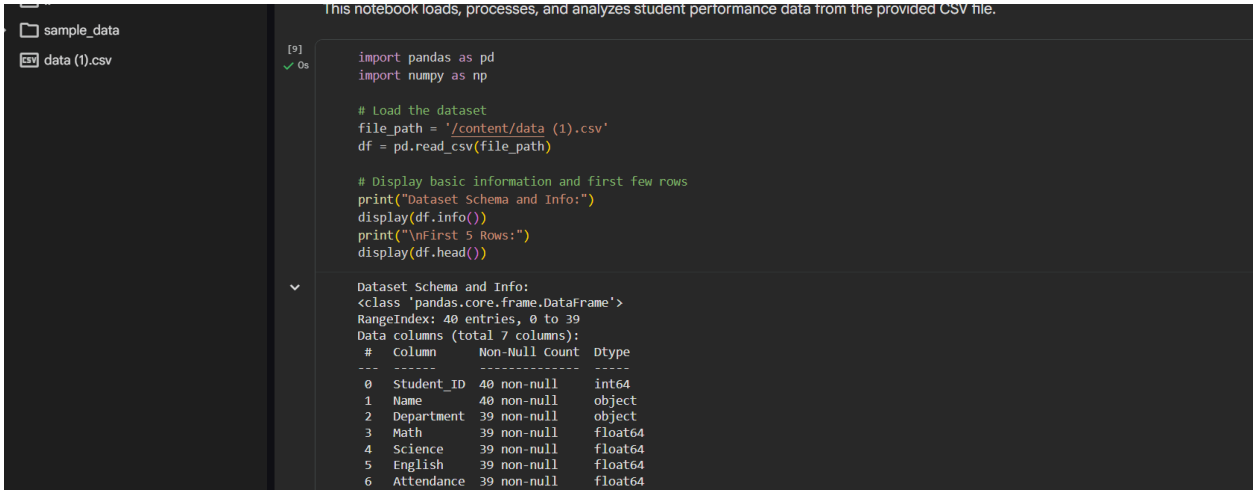
Step 3: Performance Analytics

- **Subject-Wise Analysis:** Utilized Pandas `.mean()`, `.max()`, and `.min()` methods to generate summary stats per subject.
- **Top Performers:** Identified the highest-scoring students using `df.loc[df['Average_Marks'].idxmax()]`.
- **Class Summary:** Generated overall statistics, pass percentage, and attendance analysis.

6. Key Features

- **Automated Data Ingestion:** Reads structured CSV datasets seamlessly.
- **Vectorized Processing:** Leverages Pandas and NumPy for high-performance vectorized operations without slow Python loops.
- **Dynamic Grading Logic:** Flexible rule engine for calculating grades and pass/fail eligibility.

- **Subject & Department Breakdown:** Summarizes metrics broken down by subject and academic stream.
- **Console Analytical Dashboard:** Displays clean, formatted terminal outputs for quick review.



8. Final Outcome

The project successfully automates student performance evaluation. It replaces manual calculation methods with a reliable Python system, enabling instructors to instantly generate metrics, spot struggling students, and highlight overall class trends.

9. Challenges & Problems Faced

During the implementation and testing phase, several technical and structural challenges were encountered and resolved:

1. File Path and FileNotFoundError Issues

- **Problem:** Executing `main.py` initially threw a `FileNotFoundError` because the script could not find `students.csv` when executed from different terminal working directories.

- **Solution:** Used Python's built-in `os` module (`os.path.dirname(__file__)`) to compute absolute path references relative to the script location, ensuring consistent file loading regardless of the working directory.

2. Multi-Condition Vectorized Pass/Fail Logic

- **Problem:** Standard Python `if/else` conditions failed to process Pandas series directly, causing `ValueError: The truth value of a Series is ambiguous` when checking if a student passed all individual subjects.
- **Solution:** Used NumPy's `np.where()` combined with bitwise operators (`&`) across Pandas logical conditions (e.g., `(df['Math'] >= 40) & (df['Science'] >= 40) & ...`) to achieve efficient, row-by-row vectorized evaluation.

3. Missing or Inconsistent Data Handling

- **Problem:** Test datasets with empty cells caused Pandas to cast numerical columns into float data types or caused calculations like `.sum()` to yield unexpected results.
- **Solution:** Added data cleaning steps using `df.fillna()` and validated column data types using `.astype(int)` prior to running arithmetic operations.

4. Floating Point Precision in Output Summaries

- **Problem:** Averages generated by Pandas resulted in long decimal places (e.g., `76.41999999999999`), which made terminal outputs visually messy.
- **Solution:** Applied numerical rounding (`.round(2)`) and used formatted string literals (f-strings with `:.2f` specifiers) to format clean output displays.

```
        total_marks = np.sum(marks)
        avg_marks = np.mean(marks)

        # Determine overall status (Fail if any academic subject is below passing_score)
        any_failed = any(row[sub] < passing_score for sub in subject_cols if pd.notnull(row[sub]))
        status = 'Fail' if any_failed else 'Pass'

        # Grading Rule
        if avg_marks >= 90:
            grade = 'A+'
        elif avg_marks >= 80:
            grade = 'A'
        elif avg_marks >= 70:
            grade = 'B'
        elif avg_marks >= 60:
            grade = 'C'
        elif avg_marks >= 50:
            grade = 'D'
        else:
            grade = 'F'
            status = 'Fail'

        return total_marks, avg_marks, grade, status

    # Exclude non-academic and attendance columns
    exclude_cols = ['id', 'name', 'student_id', 'roll_no', 'class', 'gender', 'age', 'attendance']
    subject_cols = [col for col in df.columns if col.lower() not in exclude_cols and np.issubdtype(df[col].dtype, np.number)]

    print(f"Detected academic subject columns for analysis: {subject_cols}")
```

```
subject_df = pd.DataFrame(subject_summary).T
print("Subject-wise Statistics:")
display(subject_df)

# Top performing student(s)
top_student_idx = df['Average Marks'].idxmax()
top_student = df.loc[top_student_idx]
print(f"\nTop Performing Student: {top_student.get('Name', top_student.get('name', 'N/A'))} with an Average of {top_student['Averag
```

```
=== Overall Class Performance Summary ===
Highest Average Score: 95.75%
Lowest Average Score:  0.00%
Class Average Score:   75.39%
========================================

Subject-wise Statistics:
```

	Highest	Lowest	Average
Math	95.00	40.0	74.77
Science	95.00	45.0	74.54
English	96.00	42.0	76.03
Total Marks	383.00	0.0	301.58
Average Marks	95.75	0.0	75.39

```
Top Performing Student: Sneha Iyer with an Average of 95.75% (Grade: A+)
```